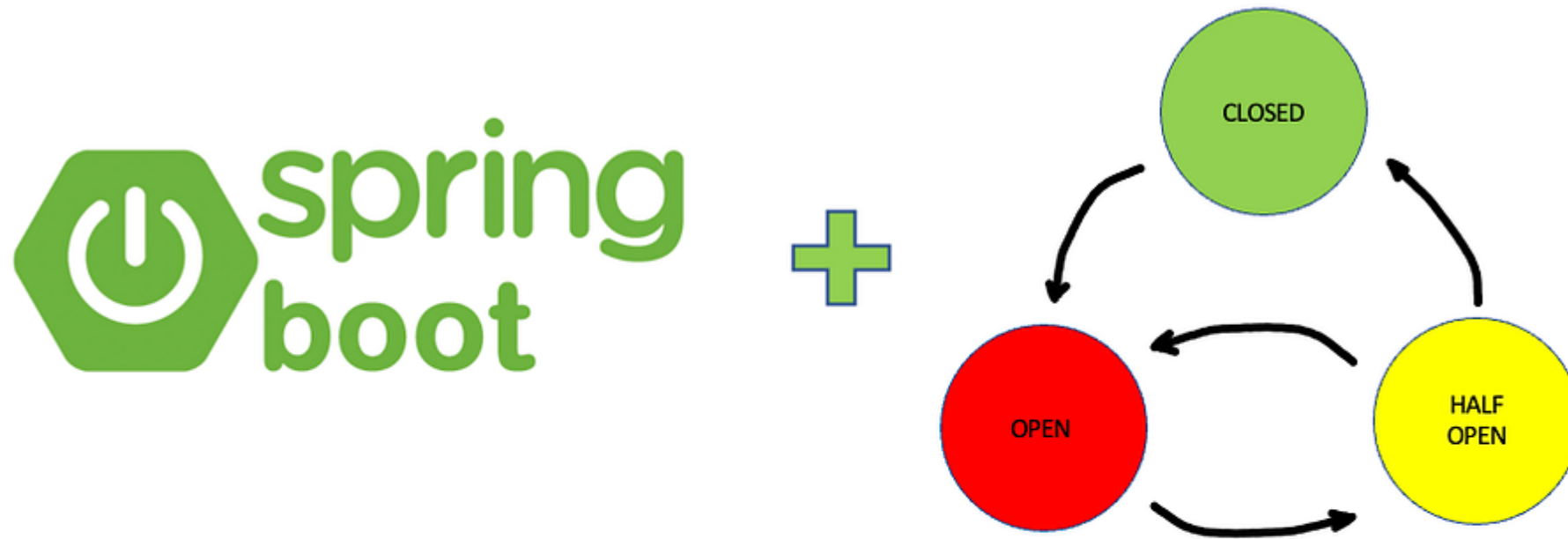


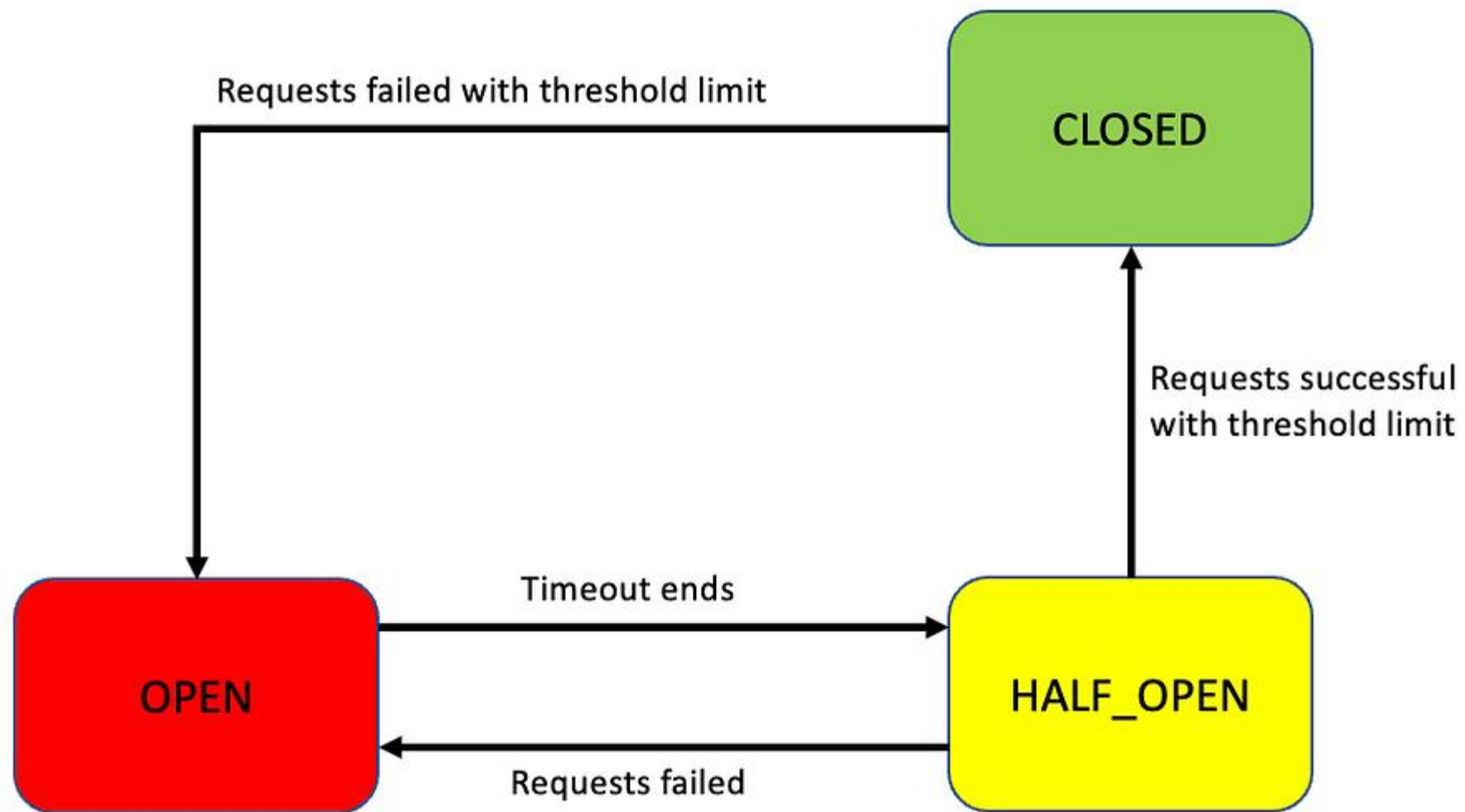
Circuit breaker

Circuit Breaker has three states: **Closed**, **Open**, and **Half_Open**.



Circuit Breaker Pattern

- pattern is to prevent calling a microservice in case of abnormal behavior in that service (service down or timeout...).
- This idea can help the client's side ignore handling failed requests and give the service some time to recover.



resilience4j-circuitbreaker configurations.

- **Resilience4j** is a lightweight, fault-tolerance library designed specifically for Java applications.
- It provides various resilience patterns, including circuit breakers, rate limiters, retries, timeouts, and bulkheads.
- Resilience4j was created as a more modular, flexible alternative to Netflix's Hystrix, allowing developers to select and configure only the resilience mechanisms they need.
-

- **sliding-window-type**: The number of requests is recorded and aggregated in the last “sliding-window-size” seconds.
- **failure-rate-threshold**: Trigger Circuit Breaker if at least “failure-rate-threshold” of requests have failed.
- **minimum-number-of-calls**: Records at least “minimum-number-of-calls” requests in the last “sliding-window-size” seconds before calculating the failure rate.
-

- **automatic-transition-from-open-to-half-open-enabled:** After “wait-duration-in-open-state” seconds, Circuit Breaker will automatically transition from open to half-open state.
- **wait-duration-in-open-state:** If triggered, wait at least “wait-duration-in-open-state” seconds before allowing more calls.
- **permitted-number-of-calls-in-half-open-state:** After “wait-duration-in-open-state” time has passed, allow another “permitted-number-of-calls-in-half-open-state” requests and wait for them to calculate the failure rate again.
- **sliding-window-size:** Record the result of the last “sliding-window-size” seconds.
-

Add dependency in productorder

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-actuator</artifactId>  
</dependency>
```

```
<dependency>  
    <groupId>io.github.resilience4j</groupId>  
    <artifactId>resilience4j-spring-boot3</artifactId>  
</dependency>
```


- start eureka-server, product, api-gatey, configserver from microservice

Add values in productorder application.yml

resilience4j:

 circuitbreaker:

 instances:

 PRODUCT:

 sliding-window-type: COUNT_BASED

 failure-rate-threshold: 50

 minimum-number-of-calls: 5

 automatic-transition-from-open-to-half-open-enabled: true

 wait-duration-in-open-state: 5s

 permitted-number-of-calls-in-half-open-state: 3

 sliding-window-size: 10

 register-health-indicator: true

 server:

 port: 5050

 spring:

 application:

 name: product-feign-client

 management:

 endpoint:

 health:

 show-details: always

 endpoints:

 web:

 base-path: /actuator

 exposure:

 include:

 - health

 - circuitbreaker

 health:

 circuitbreakers:

 enabled: true

- **Implement Circuit Breaker:** We'll apply the circuit breaker on the `getUserData` method and include a fallback method for graceful degradation.

`@CircuitBreaker(name = " PRODUCT ", fallbackMethod = "fallbackProductData")` wraps the `getUserData` method with a circuit breaker.

- `name = " PRODUCT "` links to the configuration for this circuit breaker in the `application.yml` file.
- `fallbackProductData` provides a default response if the circuit is open or an error occurs, allowing the application to continue running smoothly.

```
@Service
//@ComponentScan(value="com.example.feign")
public class ProductOrderService1 {

    @Autowired
    ProductFeign productFeign;

    public ResponseEntity<List<ProductDTO>>
    getAllProducts() {
        try {
            return productFeign.getAllProducts();
        } catch (Exception e) {
            return new ResponseEntity(e.getMessage(),
            HttpStatus.NOT_FOUND);
        }
    }
}
```

```
@CircuitBreaker(name = "PRODUCT", fallbackMethod =
"fallbackProductData")
{
    public ResponseEntity<ProductDTO> getProductById(int id)
    {
        return productFeign.getProductById(id);
    }

    public ResponseEntity<ProductDTO>
    fallbackProductData(Throwable t) {
        System.out.println("Fallback triggered for
callExternalService: " + t.getMessage());
        return new ResponseEntity("Product service is not
responding properly", HttpStatus.NOT_FOUND);
    }
}
```

Working with circuit breaker

- Start application productorder
- Check <http://localhost:5050/productorder/52>
- Check localhost:5050/actuator/health

it will show product{

"state":closed

}

now stop product

then

call <http://localhost:5050/productorder/52> API 3 times ,

then refresh the actuator link localhost:5050/actuator/health

now Circuit Breaker was triggered.

the reason is that failureRate is now greater than failure-rate-threshold.

- Wait for 5 seconds, refresh the actuator link `localhost:5050/actuator/health`, and well see Circuit

Breaker now is in HALF_OPEN state.

- During HALF_OPEN state, it allows permitted-number-of-calls-in-half-open-state requests (We configured its value as 3).
- then calculates the failure rate again, If the failure rate is still greater than failure-rate-threshold, Circuit Breaker will be triggered again. Continue calling order-service API 3 time

`http://localhost:5050/productorder/52`, then refresh the actuator link `localhost:5050/actuator/health`

`"status": "CIRCUIT_OPEN", // // CircuitBreaker is triggered again`

`"state": "OPEN"`

Now run product, then continue calling productorder API 3 times
<http://localhost:5050/productorder/52>
, refresh the actuator link
localhost:5050/actuator/health.
See Circuit Breaker was closed.